

Which Rewrite Made My Model Slower?

Finding and Blaming Bad Graph Rewrites in TorchInductor — Midterm Progress Report (S598APE)

Chidera Ibe

October 8, 2026

1 Summary

The proposal promised (a) a measurement harness that runs TorchInductor on real models with its graph rewrites turned on and off, one at a time and in combinations, (b) an attribution tool that takes a measured slowdown and names the rewrite, or pair of rewrites, that caused it, and (c) as a stretch goal, learned guards. This report covers proposal items 1–5: the corpus and rewrite switches (item 1), the measurement harness (item 2), the first batch of tests (item 3), the interim results (item 4), and a working first version of the attribution tool (item 5).

Everything described here exists as code in the `rewrite_blame` package, is driven from a command line, and is exercised by 100 automated tests that pass on both machines used. The main findings so far:

- TorchInductor 2.14.1 exposes **227 individually switchable pattern-matcher rules** plus 26 configuration flags and 16 opt-in rewrite passes, for a universe of 269 switches. Each pattern rule is switched without modifying PyTorch, by wrapping the rule’s `extra_check` hook, and the harness records how often each rule fired or was suppressed in every compile.
- The attribution tool, delta debugging over *signed* switch changes with a timing judge, recovers planted culprits in all tests, including a genuine **pair interaction** that exists in Inductor today: attention-fusion rules 11 and 12 match the same graph, so disabling either one alone changes nothing, and only disabling both changes the compiled program (Section 5).
- The same rewrite can have the opposite sign on different hardware: Inductor’s attention fusion is a 36% win on the Apple M4 Pro and an 8% *loss* on the 4-core Xeon VM, and the tool blames the right rule in both directions (Section 5). On the decode-shaped MLP the opt-in `decompose_mm_pass` is a 15% loss, blamed alone among 16 passes switched on together; on ResNet-18 a lost layout optimisation (11%) is blamed alone among six changed flags. Each attribution took 5–16 judge evaluations and under three minutes.
- On the models measured so far, most individual rewrites do not change the generated program at all (Section 4); the ones that do are the pattern rules that actually fire, the master pass switches, freezing, and a handful of scheduler knobs. This is useful: the attribution search only has to consider the rules that fired, which is a short list per model.

One deviation from the proposal. No GPU was available (neither the course VM nor my laptop has one), so every measurement in this report uses Inductor’s **CPU C++/OpenMP backend** rather than Triton. The rewrite passes under study (joint-graph, post-grad, pre-grad pattern matching, decompositions, freezing) are the same on both backends; what differs is the kernel generator. The code-statistics module parses both the C++ and the Triton wrapper formats

and is unit-tested against both, so a GPU run later is a change of device, not of tooling. Two very different CPUs (an Apple M4 Pro and an x86 Xeon with AVX-512 and oneDNN) stand in for the proposal’s “two GPUs” repeatability check.

2 What was built

2.1 Rewrite switches (proposal item 1b)

Inductor applies its rewrites in three places: *pre-grad* FX passes on the captured graph, *joint-graph* passes after AOTAutograd, and *post-grad* passes before lowering; most of them are instances of a generic `PatternMatcherPass`, each holding a list of `PatternEntry` rules keyed by the operator they match. Reading the source showed that the pass applies a rule only if `entry.extra_check(match)` returns true. The harness therefore replaces each entry’s `extra_check` with a small gate object that (1) refuses the match when the rule’s switch is off, (2) counts a *fire* when the rule’s own check accepts and the switch is on, and (3) counts a *suppression* when the rule would have fired but was disabled. No PyTorch source is modified.

Rules are registered lazily, so discovery first compiles a tiny warm-up graph that touches attention, convolution/batch-norm, split/cat, matmul and dtype casts, then walks the `fx_passes` modules for every pass object and gates every entry. Rule names come from the entry’s handler function or the replacement’s `pattern_name` (e.g. `joint/joint_graph.patterns/_sfdp_pattern_11_inference`, `post_grad/pass_patterns[1]/reciprocal_sqrt_to_rsqrt`). Three further kinds of switch cover rewrites that are not pattern entries: configuration flags (`epilogue_fusion`, `reorder_for_locality`, `freezing`, `cpp.weight_prepack`, ...), opt-in “optimus” passes enabled through Inductor’s fusion-option dictionaries (split/cat normalisation, `decompose_mm_pass`, batch fusions), and master switches for whole pass groups. A *state* is the set of switches that are on; `default` reproduces Inductor’s behaviour.

Two details mattered for correctness. First, Inductor’s on-disk FX-graph cache is keyed by its configuration but not by pattern state; a cache hit also skips every pass, which hides which rules fired. The harness therefore adds a no-op custom pass whose `uuid()` hashes the switch state into the cache key, and disables the FX-graph cache during measurements (the generated C++ kernels are still cached by source hash, so recompiles cost seconds). Second, calling Inductor’s lazy initialisers directly poisons their `functools.cache` keys and produces “duplicate pattern” errors on the next real compile; discovery must go through a real compile.

2.2 Measurement harness (item 2)

One measurement is one model under one state in a fresh subprocess, so gates, lazy state and allocator state never leak between configurations. The worker compiles the model with `torch.compile`, captures the generated wrapper code, and records:

- **generated-code statistics:** number of fused kernels (`cpp_fused_*` or `triton_*`), per-kernel input/output/in-place pointer counts, static load and store sites, vectorisation, kernel category (pointwise/reduction), extern and fallback calls (`extern_kernels.mm`, the CPU flash-attention kernel, oneDNN fused convolutions), and the bytes of intermediate buffers allocated in the wrapper (the “extra tensor written to memory” the proposal worries about);
- **Inductor’s own metrics** (bytes accessed, IR nodes before fusion, loop reorderings) and the `counters["inductor"]` deltas;
- **which rules fired or were suppressed** and the post-grad op histogram, used to verify that a switch really changed the graph;

- **correctness** against eager execution (`allclose`, `rtol/atol` 10^{-3}) under every state, so a “fast” configuration is never a wrong one;
- **time**: fixed thread count, 10 warm-up calls, then 5×10 timed calls with per-call medians, MAD, and percentiles.

Results are stored in SQLite keyed by model, machine fingerprint, state hash and timing protocol, so attribution never measures the same configuration twice; re-running an attribution is served entirely from the store.

Noise threshold. The baseline state is measured in seven separate processes. The slowdown threshold is $\tau = \max(3 \cdot \text{MAD}(\text{medians}), 1\% \text{ of the centre})$; the report also records the largest difference seen between two *identical* runs, which is the false-positive scale the threshold must exceed. Table 3 lists the measured values.

2.3 Attribution tool (item 5, started)

Given a fast state F and a slow state S , the candidate causes are the signed changes that turn F into S (`+id` = turn on, `-id` = turn off); this covers both “a new rewrite hurt” and “a rewrite that used to help was lost”. The tool runs Zeller and Hildebrandt’s *ddmin* over that change set. The judge says *slow* when the median time of F with the candidate changes applied exceeds the fast reference by more than τ ; a candidate landing within $\pm 50\%$ of τ is measured once more and the two medians averaged. The result is 1-minimal: removing any single culprit makes the slowdown disappear. A pair check then measures each culprit alone and reports an *interaction* when none reproduces the slowdown by itself. The report names the culprits and prints the kernel-level difference between F and F +culprits: kernel count and names, extern calls, intermediate bytes, and which rules changed their firing. A second judge type uses a deterministic compile-time metric (kernel count, allocation bytes) and is what the tests on planted slowdowns use, so they are immune to timing noise.

2.4 Model corpus (item 1a)

Table 1 lists the 24 registered models: six hand-written graphs that each plant one rewrite family, six torchvision models, and twelve HuggingFace configurations including LLM prefill and single-token (decode) shapes. Models are built from fixed configs with random weights, so nothing is downloaded and timings do not depend on weights. Decode graphs are single-token forwards without a KV cache for now.

3 Testing (item 3)

Tests are split into *unit* tests (no compilation; `tests/test_u_*.py`) and *Inductor* tests that compile small graphs (`tests/test_i_*.py`); Table 2 counts them. Highlights:

- **ddmin on synthetic cost models**: single culprit, pair-only culprit (flagged as an interaction), two independent culprits (1-minimal result), three-way interaction, a masking rewrite that speeds up, a noisy judge (18/20 seeds correct at $\tau = 3\sigma$), 25 random cost models checked for “slow and 1-minimal”, memoisation, and the logarithmic number of judge calls (a culprit among 256 changes in ≤ 40 evaluations).
- **counters**: the code-statistics parser on hand-written C++ and Triton wrapper fixtures with known kernel counts, pointer counts, loads/stores, extern calls, allocation bytes including a symbolic shape, and the diff that ignores kernel numbering.

model	family	size	description
attention_block	toy	tiny	manual attention (B=4,T=128,C=512,H=8)
conv_bn_relu_stack	toy	tiny	3x conv3x3-bn-relu on 8x32x56x56
split_cat_mlp	toy	tiny	split->4 linears->cat->linear
norm_mlp	toy	tiny	layernorm/gelu MLP with pointless convert/view and 1/sqrt
decode_mlp	toy	tiny	M=1 gated MLP (LLM decode shape)
cat_slice_cat	toy	tiny	cat/slice/split_with_sizes/cat chains
resnet18	vision	small	torchvision resnet18, batch 8, 224x224
resnet50	vision	medium	torchvision resnet50, batch 4, 224x224
mobilenet_v3_small	vision	small	torchvision mobilenet_v3_small, batch 8, 224x224
efficientnet_b0	vision	medium	torchvision efficientnet_b0, batch 4, 224x224
convnext_tiny	vision	medium	torchvision convnext_tiny, batch 4, 224x224
vit_b_16	vision	large	torchvision vit_b_16, batch 4, 224x224
bert_base	hf	medium	BERT-base encoder, batch 8, seq 128, attn=default
bert_base_eagerattn	hf	medium	BERT-base encoder, batch 8, seq 128, attn=eager
distilbert	hf	small	DistilBERT encoder, batch 8, seq 128, attn=default
gpt2_prefill	hf	medium	GPT-2 small prefill, batch 4, seq 128, attn=default
gpt2_prefill_eagerattn	hf	medium	GPT-2 small prefill, batch 4, seq 128, attn=eager
gpt2_decode	hf	small	GPT-2 small single-token step (no KV cache), batch 4, seq 1, attn=default
t5_small_encoder	hf	small	T5-small encoder, batch 8, seq 128, attn=default
smollm_135m_prefill	hf	medium	SmolLM-135M (LLaMA arch) prefill, batch 1, seq 128, attn=default
smollm_135m_prefill_eagerattn	hf	medium	SmolLM-135M (LLaMA arch) prefill, batch 1, seq 128, attn=eager
smollm_135m_decode	hf	small	SmolLM-135M single-token step (no KV cache), batch 1, seq 1, attn=default
qwen2_0_5b_prefill	hf	large	Qwen2.5-0.5B prefill, batch 1, seq 128, attn=default
qwen2_0_5b_decode	hf	medium	Qwen2.5-0.5B single-token step (no KV cache), batch 1, seq 1, attn=default

Table 1: Model corpus (random-init from fixed configs).

- **noise threshold**: MAD with floor, quiet-machine floor, re-measure band of the timing judge, serialisation.
- **planted rewrites** (compiled): for each toy graph, the expected rule fires by default, its switch suppresses it, the post-grad graph changes in the predicted way (e.g. `rsqrt` becomes `sqrt+reciprocal`; the dtype round trip survives without `pointless_convert`; SDPA disappears and two `bmm` calls plus a softmax kernel appear without the attention rules; `decompose_mm_pass` removes `mm` calls on the decode MLP; freezing folds batch-norm into the convolutions), and outputs match eager under both states.
- **pipeline**: subprocess isolation, store caching (second call makes no new measurement), failure reporting, the planted sfdp {11,12} interaction recovered by attribution with a metric judge in fewer compiles than leave-one-out, a single config culprit hidden among no-op changes, switch-effect classification, and determinism of the switch registry across processes.

test file	tests
test_i_discover.py	17
test_i_pipeline.py	9
test_i_planted.py	9
test_u_attribute.py	40
test_u_codegen_stats.py	6
test_u_judge_stats.py	8
test_u_store.py	3
test_u_switches.py	8
total	100

Table 2: Automated tests (unit tests run in well under a second; Inductor tests compile toy graphs).

4 Results so far (item 4)

4.1 Noise

model	machine	runs	centre (ms)	MAD (ms)	τ (ms)	τ (%)	max $ \Delta $ (ms)
attention_block	Mac	7	0.926	0.006	0.017	1.8	0.029
bert_base_eagerattn	Mac	7	84.363	0.968	2.905	3.4	1.708
decode_mlp	Mac	7	0.194	0.004	0.011	5.5	0.046
gpt2_prefill	Mac	7	62.447	0.600	1.800	2.9	1.125
norm_mlp	Mac	7	1.058	0.006	0.018	1.7	0.047
resnet18	Mac	7	26.118	0.192	0.576	2.2	0.451
attention_block	VM	7	8.526	0.066	0.198	2.3	0.320
norm_mlp	VM	7	12.478	0.052	0.155	1.2	1.660

Table 3: Baseline noise from repeated identical runs in separate processes (Mac: Apple M4 Pro, 8 threads; VM: course VM, 4-core Xeon Silver 4216). Last column: largest difference between two identical runs.

4.2 Does flipping a switch change the program?

For each model the harness compiles the default state, then toggles every candidate switch (every rule that fired, every config flag, every opt-in option) and classifies the effect: *changes graph* (different kernels, extern calls, allocation or post-grad ops), *fires only* (a different rule fired but the program is the same), or *no effect*.

model	kernels	rules fired	candidates (pat/cfg/opt)	changes graph	fires only	no effect	errors
attention_block	1	4	4/26/16	5	6	35	0
bert_base_eagerattn	5	4	4/26/16	5	5	36	0
decode_mlp	1	0	0/26/16	4	0	38	0
gpt2_prefill	10	3	3/26/16	7	4	34	0
norm_mlp	3	4	4/26/16	8	2	36	0
resnet18	14	1	1/26/16	4	3	36	0
attention_block	1	4	4/26/16	5	6	35	0
norm_mlp	3	4	4/26/16	8	2	36	0

Table 4: Switch verification: effect of toggling each candidate switch on the compiled program. A pattern rule that never fired cannot change anything when turned off, so only fired rules are candidates.

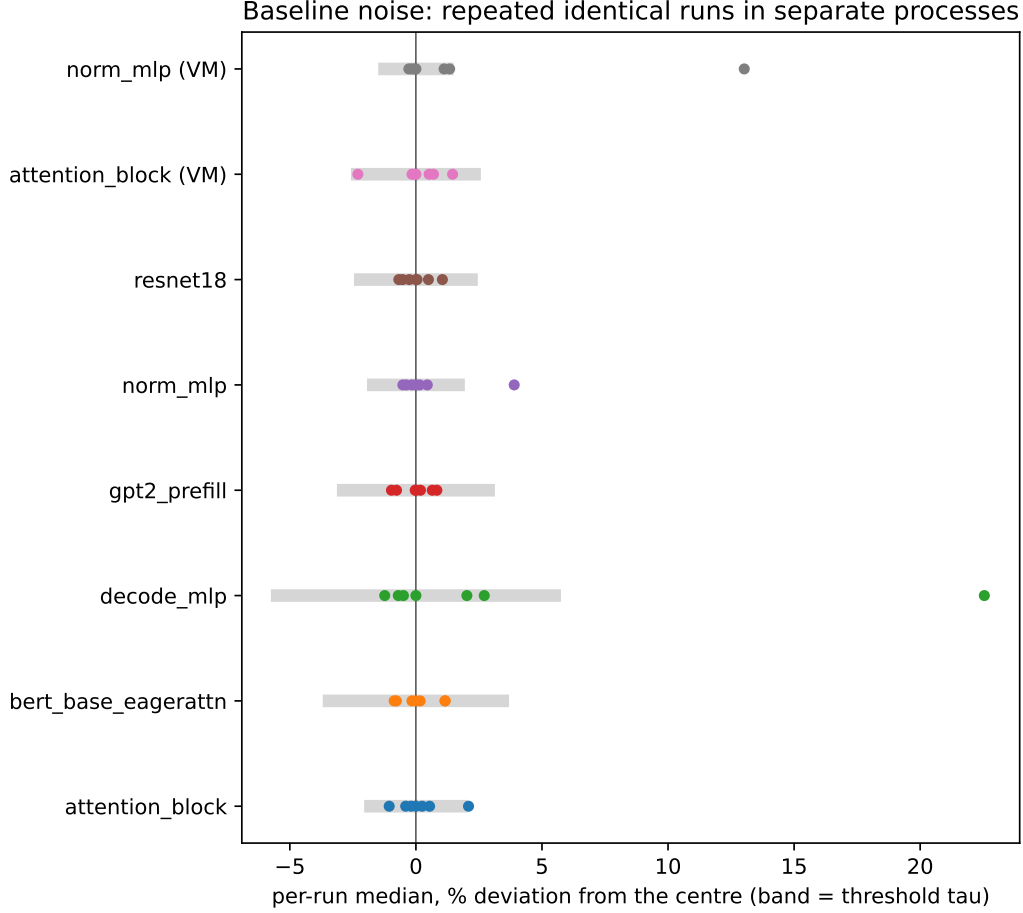


Figure 1: Per-run medians of identical baseline runs, as deviation from the centre; the grey band is τ .

4.3 Leave-one-out timing sweeps

For every graph-changing switch of Table 4, the model is re-timed with that one switch toggled and compared with the baseline against τ . The per-model tables and figures below are generated from the stored measurements; three observations stand out.

- **A rewrite that hurts on its own.** On the single-token MLP (`decode_mlp`, the shape an LLM decode step produces), turning on Inductor’s opt-in `decompose_mm_pass`, which rewrites a skinny $1 \times K$ matmul into a broadcast multiply and a sum, makes the step **15% slower** on the M4 Pro even though it removes two `mm` calls: the fused pointwise kernel reads the whole weight matrix with worse arithmetic intensity than the BLAS call it replaced. This is exactly the “looks like a win locally, loses globally” pattern the project is about, and it is a candidate guard target for the stretch goal.
- **Rewrites whose loss hurts.** Disabling the attention fusion rules on the attention block costs 33%; disabling channels-last layout optimisation on ResNet-18 costs 11%; disabling in-place buffer reuse on the layer-norm MLP costs 4%; disabling uniform-value constant folding costs 2.6%. These are the rewrites an engineer would want named when an upgrade silently stops applying them.

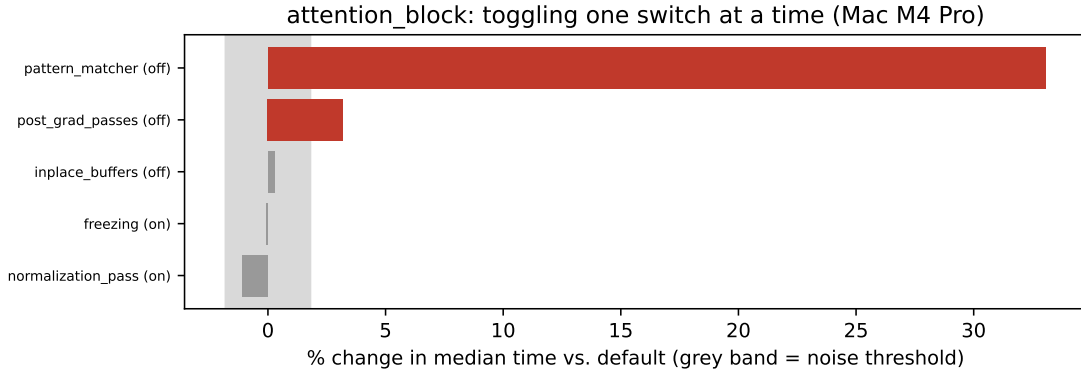
- **Most switches are no-ops for a given model.** Of the roughly 45 candidate switches per model, only 4–8 change the compiled program at all (Table 4), and only a subset of those move the time beyond τ . The attribution search benefits twice: candidates are few, and the no-op ones are eliminated in the first ddmin halvings.

attention_block on the Mac (M4 Pro, 8 threads)

Baseline 0.926 ms, $\tau = 0.017$ ms (1.8%). 5 graph-changing switches were toggled one at a time: 2 made the model slower than τ , 0 made it faster, 3 stayed within noise. Slower: **master/pattern_matcher** (off, +33.1%); **master/post_grad_passes** (off, +3.2%).

switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
master/pattern_matcher	off	1.232	+33.1	4	slower
master/post_grad_passes	off	0.955	+3.2	2	slower
optimus/normalization_pass	on	0.916	-1.1	1	
sched/inplace_buffers	off	0.928	+0.3	1	
lowering/freezing	on	0.925	-0.0	1	

Table 5: Leave-one-out sweep, attention_block, Mac (M4 Pro, 8 threads).

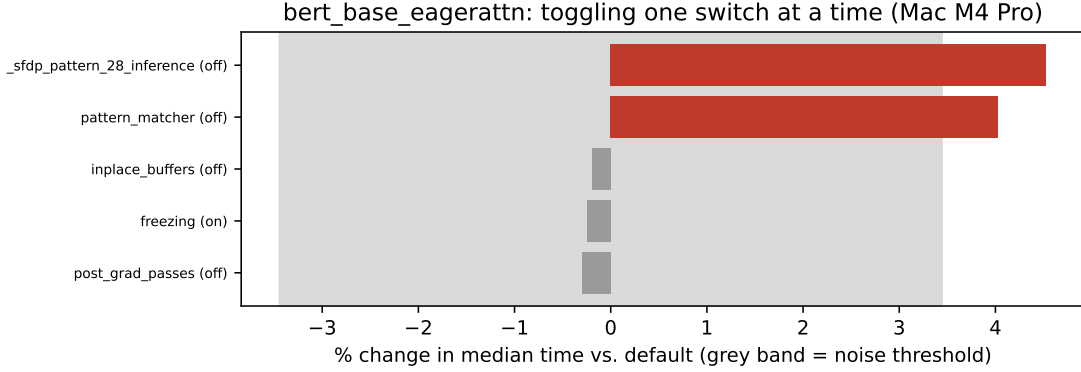


bert_base_eagerattn on the Mac (M4 Pro, 8 threads)

Baseline 84.363 ms, $\tau = 2.905$ ms (3.4%). 5 graph-changing switches were toggled one at a time: 2 made the model slower than τ , 0 made it faster, 3 stayed within noise. Slower: **_sfdp_pattern_28_inference** (off, +4.5%); **master/pattern_matcher** (off, +4.0%).

switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
_sfdp_pattern_28_inference	off	88.183	+4.5	7	slower
master/pattern_matcher	off	87.762	+4.0	7	slower
master/post_grad_passes	off	84.110	-0.3	5	
lowering/freezing	on	84.159	-0.2	5	
sched/inplace_buffers	off	84.197	-0.2	5	

Table 6: Leave-one-out sweep, bert_base_eagerattn, Mac (M4 Pro, 8 threads).

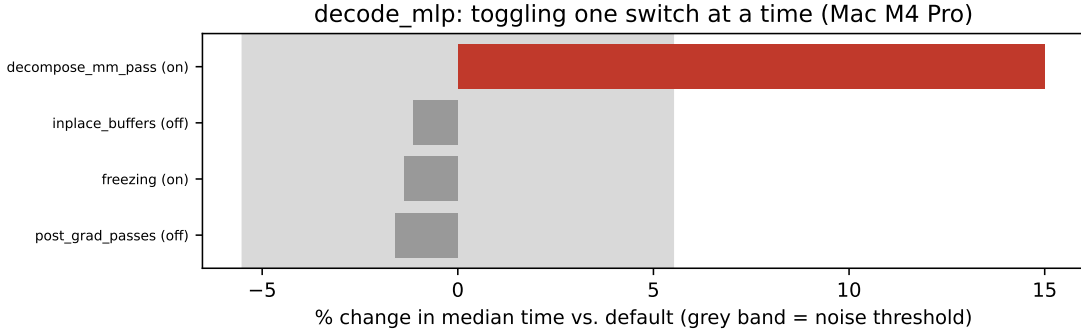


decode_mlp on the Mac (M4 Pro, 8 threads)

Baseline 0.194 ms, $\tau = 0.011$ ms (5.5%). 4 graph-changing switches were toggled one at a time: 1 made the model slower than τ , 0 made it faster, 3 stayed within noise. Slower: `optimus/decompose_mm_pass` (on, +15.0%).

switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
optimus/decompose_mm_pass	on	0.223	+15.0	1	slower
master/post_grad_passes	off	0.191	-1.6	1	
lowering/freezing	on	0.191	-1.4	1	
sched/inplace_buffers	off	0.191	-1.1	1	

Table 7: Leave-one-out sweep, decode_mlp, Mac (M4 Pro, 8 threads).



gpt2_prefill on the Mac (M4 Pro, 8 threads)

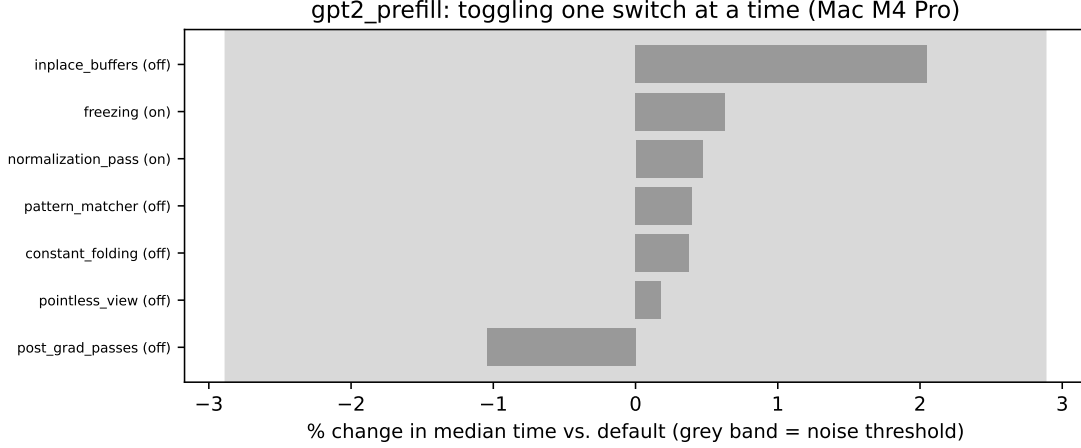
Baseline 62.447 ms, $\tau = 1.800$ ms (2.9%). 7 graph-changing switches were toggled one at a time: 0 made the model slower than τ , 0 made it faster, 7 stayed within noise.

norm_mlp on the Mac (M4 Pro, 8 threads)

Baseline 1.058 ms, $\tau = 0.018$ ms (1.7%). 8 graph-changing switches were toggled one at a time: 2 made the model slower than τ , 0 made it faster, 6 stayed within noise. Slower: `sched/inplace_buffers` (off, +4.3%); `joint/constant_folding` (off, +2.6%).

switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
sched/inplace_buffers	off	63.725	+2.0	9	
master/post_grad_passes	off	61.797	-1.0	10	
lowering/freezing	on	62.840	+0.6	10	
optimus/normalization_pass	on	62.741	+0.5	10	
master/pattern_matcher	off	62.696	+0.4	10	
joint/constant_folding	off	62.683	+0.4	10	
pointless_view	off	62.559	+0.2	10	

Table 8: Leave-one-out sweep, gpt2_prefill, Mac (M4 Pro, 8 threads).



resnet18 on the Mac (M4 Pro, 8 threads)

Baseline 26.118 ms, $\tau = 0.576$ ms (2.2%). 4 graph-changing switches were toggled one at a time: 1 made the model slower than τ , 0 made it faster, 3 stayed within noise. Slower: `lowering/layout_optimization` (off, +11.1%).

attention_block on the VM (Xeon, 4 threads)

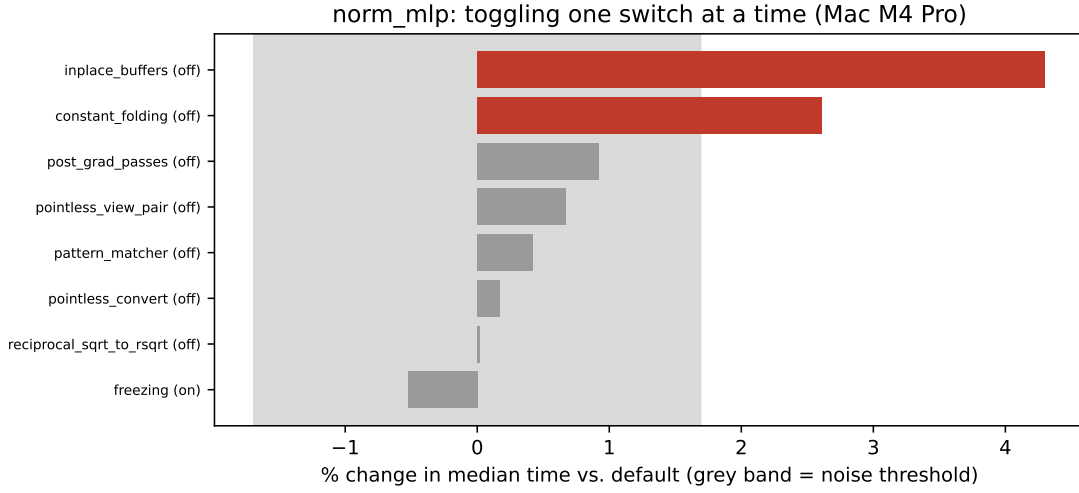
Baseline 8.526 ms, $\tau = 0.198$ ms (2.3%). 5 graph-changing switches were toggled one at a time: 0 made the model slower than τ , 4 made it faster, 1 stayed within noise. Faster: `master/pattern_matcher` (off, -10.6%); `lowering/freezing` (on, -5.3%); `sched/inplace_buffers` (off, -3.3%); `optimus/normalization_pass` (on, -2.7%).

norm_mlp on the VM (Xeon, 4 threads)

Baseline 12.478 ms, $\tau = 0.155$ ms (1.2%). 8 graph-changing switches were toggled one at a time: 0 made the model slower than τ , 1 made it faster, 7 stayed within noise. Faster: `lowering/freezing` (on, -46.6%).

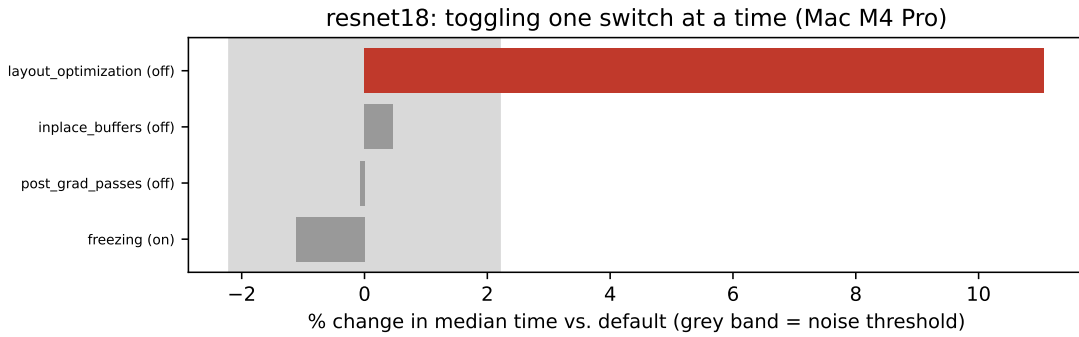
switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
sched/inplace_buffers	off	1.103	+4.3	3	slower
joint/constant_folding	off	1.085	+2.6	3	slower
master/post_grad_passes	off	1.067	+0.9	3	
pointless_view_pair	off	1.065	+0.7	3	
lowering/freezing	on	1.052	-0.5	3	
master/pattern_matcher	off	1.062	+0.4	3	
pointless_convert	off	1.059	+0.2	3	
reciprocal_sqrt_to_rsqr	off	1.058	+0.0	3	

Table 9: Leave-one-out sweep, norm_mlp, Mac (M4 Pro, 8 threads).



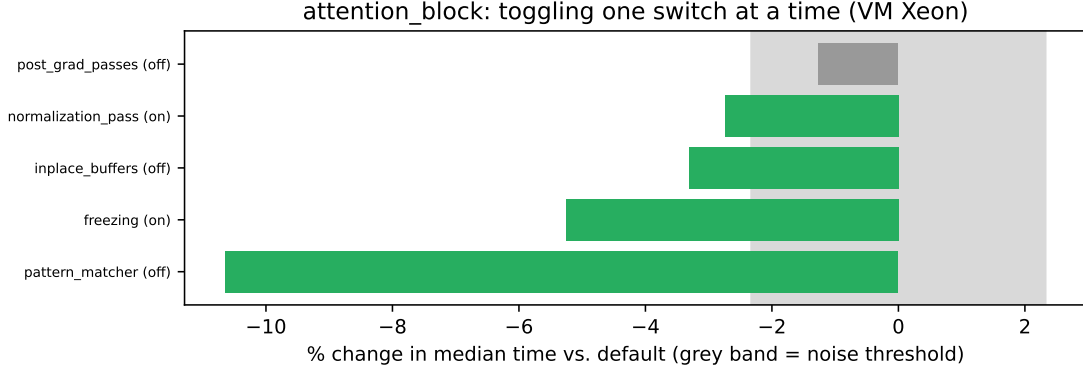
switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
lowering/layout_optimization	off	29.007	+11.1	12	slower
lowering/freezing	on	25.827	-1.1	11	
sched/inplace_buffers	off	26.239	+0.5	14	
master/post_grad_passes	off	26.099	-0.1	14	

Table 10: Leave-one-out sweep, resnet18, Mac (M4 Pro, 8 threads).



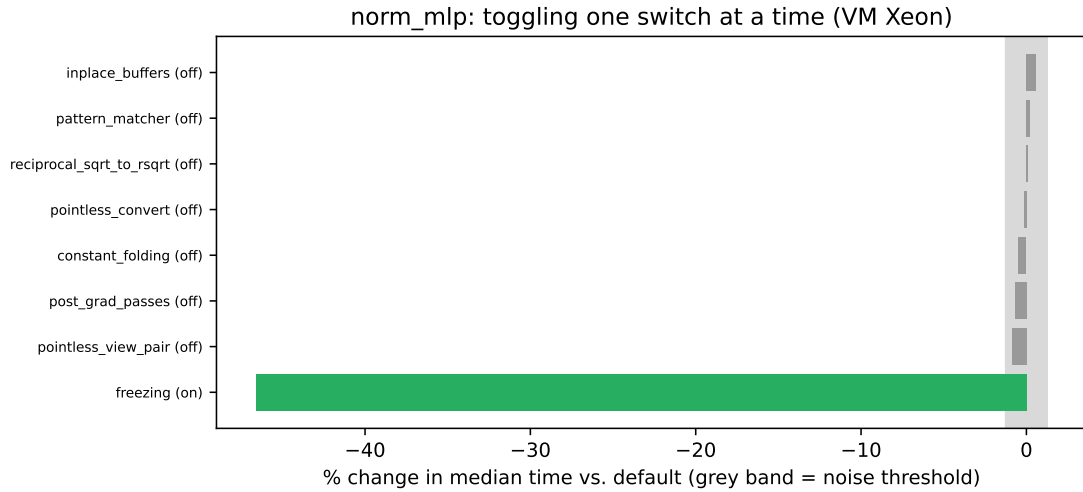
switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
master/pattern_matcher	off	7.619	-10.6	4	faster
lowering/freezing	on	8.078	-5.3	2	faster
sched/inplace_buffers	off	8.244	-3.3	1	faster
optimus/normalization_pass	on	8.292	-2.7	1	faster
master/post_grad_passes	off	8.418	-1.3	2	

Table 11: Leave-one-out sweep, attention_block, VM (Xeon, 4 threads).



switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
lowering/freezing	on	6.664	-46.6	3	faster
pointless_view_pair	off	12.369	-0.9	3	
master/post_grad_passes	off	12.391	-0.7	3	
sched/inplace_buffers	off	12.552	+0.6	3	
joint/constant_folding	off	12.418	-0.5	3	
master/pattern_matcher	off	12.507	+0.2	3	
pointless_convert	off	12.462	-0.1	3	
reciprocal_sqrt_to_rsqr	off	12.487	+0.1	3	

Table 12: Leave-one-out sweep, norm_mlp, VM (Xeon, 4 threads).



5 Attribution case studies

Each case study feeds the tool a fast state and a slow state that differ in many switches, most of them irrelevant, and asks it to name the culprits with the timing judge and the measured τ . The attention-block case is the headline result: the slow state turns off all 63 inference-time attention-fusion rules; the tool reports that exactly two of them, `_sfdp_pattern_11_inference` and `_sfdp_pattern_12_inference`, are responsible, *and that neither matters alone* (rule 12 matches the same graph when 11 is disabled, so a leave-one-out sweep would have blamed nothing). The kernel-level diff shows why the loss costs 36%: one fused kernel plus one flash-attention call become four fused kernels and two `bmm` calls, and intermediate traffic grows from 4.2 MB to 11.6 MB per step.

attention_block on the Mac (M4 Pro, 8 threads): verdict *interaction*

The slow state differs from the fast state in 63 switches. Fast 0.919 ms vs. slow 1.250 ms (+36.1%); $\tau = 0.017$ ms. `ddmin` needed 16 judge evaluations (versus 63 for leave-one-out, 1953 for all pairs) and returned the culprit set `-joint/joint_graph.patterns/_sfdp_pattern_11_inference, -joint/joint_graph.patterns/_sfdp_pattern_12_inference`. Each culprit alone was judged *fast*: the slow-down needs all of them together. **Kernel-level explanation.** fused kernels `1` $\rightarrow$ `4`, extern calls `3` $\rightarrow$ `4`, intermediate bytes `4,198,400` $\rightarrow$ `11,591,680`. Kernels gained: `cpp_fused__softmax_abs_all_amax_div_eq_mul_ne_sub_view`, `cpp_fused_clone_split_transpose_view`, `cpp_fused_clone_transpose_view`. Extern calls lost: `torch.ops.aten._scaled_dot_product_flash_attention_for_cpu`; gained: `extern_kernels.bmm` ($\times 2$). Rules suppressed in the culprit state: `_sfdp_pattern_11_inference`, `_sfdp_pattern_12_inference`.

field	value
model	attention_block (Chideras-MacBook-Pro-2)
judge	timing (median_ms)
fast reference	0.926 ms, $\tau = 0.017$ ms (1.8%)
slow state	1.250 ms
candidate changes	63
judge evaluations	16
verdict	interaction
culprits	<code>_sfdp_pattern_11_inference</code> , <code>_sfdp_pattern_12_inference</code>
kernels fast $\rightarrow$ culprits	<code>1</code> $\rightarrow$ <code>4</code>
extern calls	<code>3</code> $\rightarrow$ <code>4</code>
intermediate bytes	<code>4,198,400</code> $\rightarrow$ <code>11,591,680</code>
median ms	<code>0.919</code> $\rightarrow$ <code>1.244</code>

Table 13: Attribution summary, attention_block.

decode_mlp on the Mac (M4 Pro, 8 threads): verdict *single*

The slow state differs from the fast state in 16 switches. Fast 0.195 ms vs. slow 0.222 ms (+14.1%); $\tau = 0.011$ ms. `ddmin` needed 7 judge evaluations (versus 16 for leave-one-out, 120 for all pairs) and returned the culprit set `+optimus/decompose_mm_pass`. A single change reproduces the whole slowdown. **Kernel-level explanation.** fused kernels `1` $\rightarrow$ `1`, extern calls `3` $\rightarrow$ `0`, intermediate bytes `20,480` $\rightarrow$ `20,480`. Kernels lost: `cpp_fused_mul_silu`. Kernels gained: `cpp_fused_mm_mul_silu_t`. Extern calls lost: `extern_kernels.mm` ($\times 3$); gained: none.

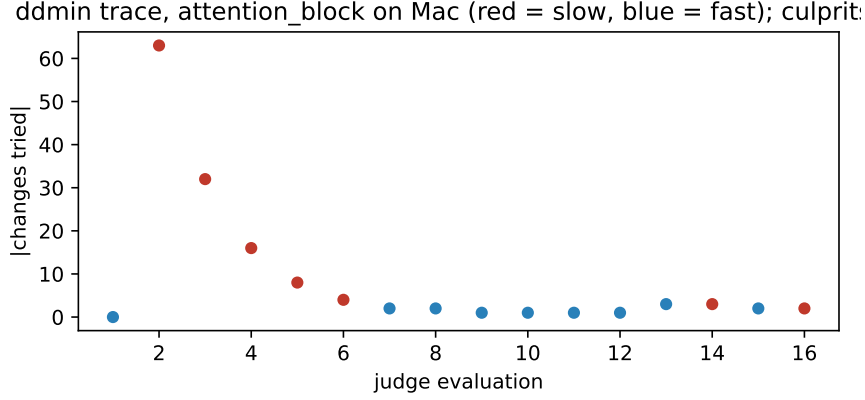


Figure 2: ddmin search trace for attention_block (Mac (M4 Pro, 8 threads)): each point is one judge evaluation.

field	value
model	decode_mlp (Chideras-MacBook-Pro-2)
judge	timing (median_ms)
fast reference	0.194 ms, $\tau = 0.011$ ms (5.5%)
slow state	0.222 ms
candidate changes	16
judge evaluations	7
verdict	single
culprits	+optimus/decompose_mm_pass
kernels fast $\rightarrow$ culprits	1 $\rightarrow$ 1
extern calls	3 $\rightarrow$ 0
intermediate bytes	20,480 $\rightarrow$ 20,480
median ms	0.195 $\rightarrow$ 0.223

Table 14: Attribution summary, decode_mlp.

norm_mlp on the Mac (M4 Pro, 8 threads): verdict *single*

The slow state differs from the fast state in 8 switches. Fast 1.061 ms vs. slow 1.099 ms (+3.6%); $\tau = 0.018$ ms. ddmin needed 8 judge evaluations (versus 8 for leave-one-out, 28 for all pairs) and returned the culprit set `-joint/constant_folding`. A single change reproduces the whole slowdown. **Kernel-level explanation.** fused kernels 3 $\rightarrow$ 3, extern calls 2 $\rightarrow$ 2, intermediate bytes 1,573,376 $\rightarrow$ 1,573,376. Kernels lost: `cpp_fused_add_gelu_mean_mul_pow`. Kernels gained: `cpp_fused_add_gelu_mean_mul_pow_reciprocal`.

resnet18 on the Mac (M4 Pro, 8 threads): verdict *single*

The slow state differs from the fast state in 6 switches. Fast 26.205 ms vs. slow 29.119 ms (+11.1%); $\tau = 0.576$ ms. ddmin needed 5 judge evaluations (versus 6 for leave-one-out, 15 for all pairs) and returned the culprit set `-lowering/layout_optimization`. A single change reproduces the whole slowdown. **Kernel-level explanation.** fused kernels 14 $\rightarrow$ 12, extern calls 21 $\rightarrow$ 21, intermediate bytes 30,052,352 $\rightarrow$ 6,470,912. Kernels lost: `cpp_fused__native_batch_norm_legit_no_training_add_convolution_relu` ($\times 7$), `cpp_fused__native_batch_norm_legit_no_training_`

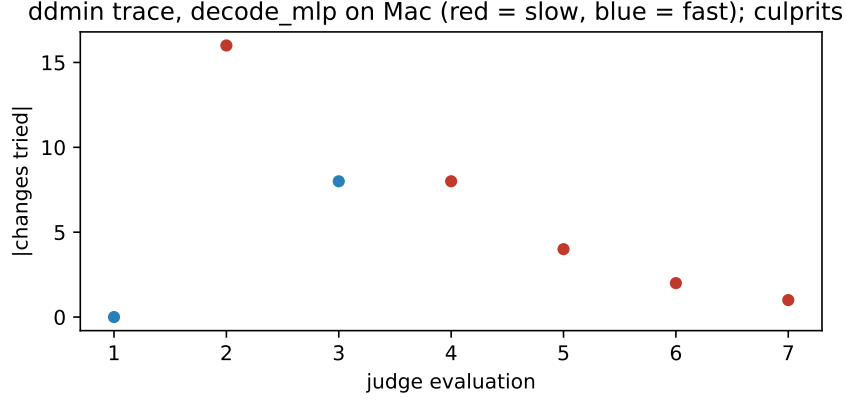


Figure 3: ddmin search trace for decode_mlp (Mac (M4 Pro, 8 threads)): each point is one judge evaluation.

field	value
model	norm_mlp (Chideras-MacBook-Pro-2)
judge	timing (median_ms)
fast reference	1.058 ms, $\tau = 0.018$ ms (1.7%)
slow state	1.099 ms
candidate changes	8
judge evaluations	8
verdict	single
culprits	-joint/constant_folding
kernels fast $\rightarrow$ culprits	3 $\rightarrow$ 3
extern calls	2 $\rightarrow$ 2
intermediate bytes	1,573,376 $\rightarrow$ 1,573,376
median ms	1.061 $\rightarrow$ 1.085

Table 15: Attribution summary, norm_mlp.

convolution_max_pool2d_with_indices_relu, cpp_fused_convolution. Kernels gained: cpp_fused__native_batch_norm_legit_no_training_add_relu ($\times 6$), cpp_fused__native_batch_norm_legit_no_training_max_pool2d_with_indices_relu.

attention_block on the VM (Xeon, 4 threads): verdict *single*

The slow state differs from the fast state in 63 switches. Fast 7.621 ms vs. slow 8.265 ms (+8.4%); $\tau = 0.162$ ms. ddmin needed 8 judge evaluations (versus 63 for leave-one-out, 1953 for all pairs) and returned the culprit set +joint/joint_graph.patterns/_sfdp_pattern_11_inference. A single change reproduces the whole slowdown. **Kernel-level explanation.** fused kernels 4 $\rightarrow$ 1, extern calls 4 $\rightarrow$ 3, intermediate bytes 11,591,680 $\rightarrow$ 4,198,400. Kernels lost: cpp_fused__softmax_abs_all_amax_div_eq_mul_ne_sub_view, cpp_fused_clone_split_transpose_view, cpp_fused_clone_transpose_view. Extern calls lost: extern_kernels.bmm ($\times 2$); gained: torch.ops.aten._scaled_dot_product_flash_attention_for_cpu.

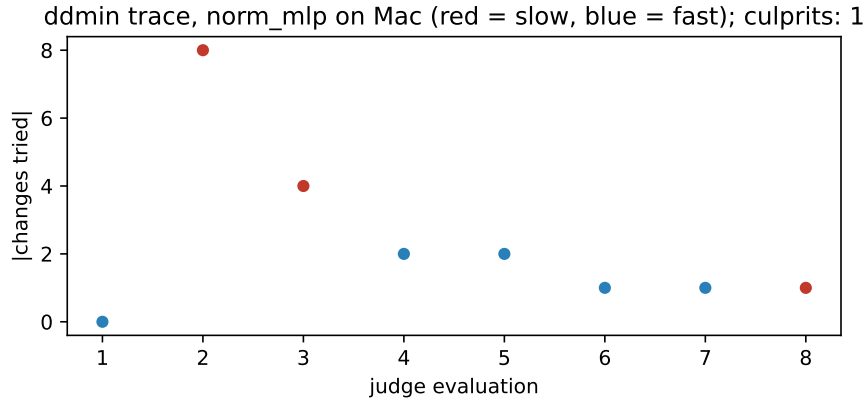


Figure 4: ddmin search trace for norm_mlp (Mac (M4 Pro, 8 threads)): each point is one judge evaluation.

field	value
model	resnet18 (Chideras-MacBook-Pro-2)
judge	timing (median_ms)
fast reference	26.118 ms, $\tau = 0.576$ ms (2.2%)
slow state	29.119 ms
candidate changes	6
judge evaluations	5
verdict	single
culprits	-lowering/layout_optimization
kernels fast $\rightarrow$ culprits	14 $\rightarrow$ 12
extern calls	21 $\rightarrow$ 21
intermediate bytes	30,052,352 $\rightarrow$ 6,470,912
median ms	26.205 $\rightarrow$ 29.007

Table 16: Attribution summary, resnet18.

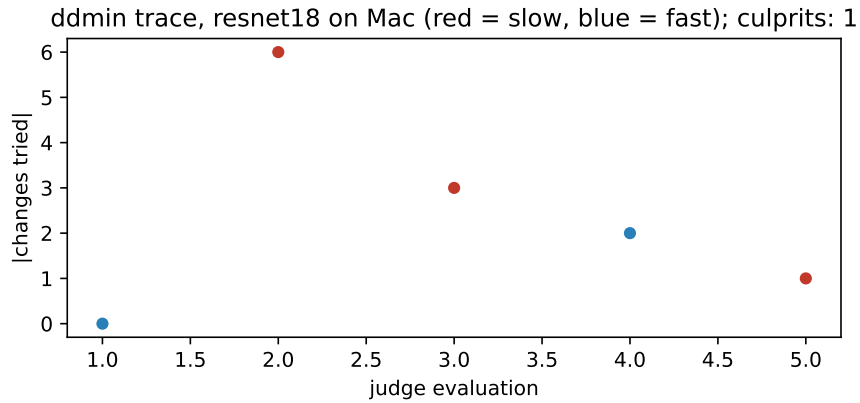


Figure 5: ddmin search trace for resnet18 (Mac (M4 Pro, 8 threads)): each point is one judge evaluation.

field	value
model	attention_block (fa26-cs598a-010)
judge	timing (median_ms)
fast reference	7.650 ms, $\tau = 0.162$ ms (2.1%)
slow state	8.265 ms
candidate changes	63
judge evaluations	8
verdict	single
culprits	_sfdp_pattern_11_inference
kernels fast $\rightarrow$ culprits	4 $\rightarrow$ 1
extern calls	4 $\rightarrow$ 3
intermediate bytes	11,591,680 $\rightarrow$ 4,198,400
median ms	7.621 $\rightarrow$ 8.326

Table 17: Attribution summary, attention_block.

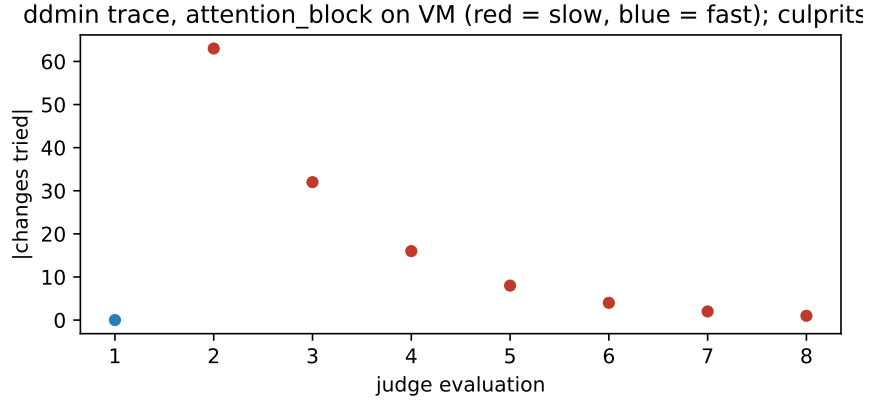


Figure 6: ddmin search trace for attention_block (VM (Xeon, 4 threads)): each point is one judge evaluation.

What ddmin does and does not promise

The layer-norm MLP case is instructive. Its sweep shows two independent causes (in-place buffer reuse, 4.3%; constant folding, 2.6%), but the attribution over eight changed flags returned only `-joint/constant_folding`. That is correct behaviour: ddmin returns a *1-minimal* set, one whose every element is necessary to cross τ , and constant folding alone already crosses it. It does not promise to list every independent cause. The practical answer, planned for item 5, is to iterate: re-run with the found culprits moved into the fast state, until the remaining difference is within noise. The sweep table already gives the full picture for single switches, and the ddmin result adds what the sweep cannot: the interactions.

The same rewrite, two machines, opposite sign

The attention block was attributed on both machines with the roles of “fast” and “slow” chosen by measurement. On the Apple M4 Pro the fused CPU flash-attention kernel that the `_sfdp` rules produce is a 36% win, so losing the rules is the slowdown and the tool blames the {11,12} pair. On the 4-core Xeon the fused kernel is **8% slower** than the unfused path (two `bmm` calls plus a fused softmax kernel), so there the rewrite itself is the slowdown, and the tool blames `+_sfdp_pattern_11_inference` as a single culprit (rule 12 is redundant once 11 is on, so turning 11 on alone already reproduces the loss). This is the clearest evidence so far for the proposal’s thesis: the rewrite is locally sensible, its value depends on what the kernel generator and the hardware do with the result, and nothing in Inductor checks. It is also the natural first target for a learned guard (stretch item 7), since the decision depends on a few cheap features (sequence length, head size, thread count, ISA). The VM is a shared host and shows occasional outlier runs (one identical-run pair differed by 1.7 ms against a MAD of 0.05 ms); the MAD-based τ is robust to such outliers, and every “slow” verdict in the trace was reproduced by at least three independent measurements.

6 Status against the proposal and plan to the final report

proposal item	status
1. Corpus (10–15 models) and a switch per rewrite rule, verified to change the graph	Done. 24 models registered, 269 switches, verification tool classifies each switch’s effect per model.
2. Measurement harness: cached compiles, kernel and byte counters, timing with warm-up/trials, noise threshold	Done on the CPU backend; counters also parse Triton output (unit-tested).
3. Testing task 1: ≥ 30 unit tests with planted slowdowns, counter and threshold tests	Done, growing: 100 tests (Table 2).
4. Interim report: harness on 3–4 models with one clean attribution	Done: this report.
5. Attribution tool: ddmin over rule sets, timing judge, pair check, kernel-level diff	Working first version; needs more real-model case studies and a sensitivity study of τ .
6. Testing task 2: full corpus, two largest models, repeatability across days and machines	Started: sweeps run on both machines; the largest models (ViT-B/16, Qwen2.5-0.5B) are registered but not yet measured.
7. Stretch: learned guards	Not started.
8. Final report, README, cleanup	README exists.

Risks and next steps. (1) Timing noise on the shared 4-core VM is the main threat to attribution; the mitigation is the measured τ , the re-measure band, and running the sweeps on both machines. (2) Most individual rewrites are no-ops on a given model, so slowdowns on real models will come from a few rules (attention fusion, freezing/oneDNN fusion, `decompose_mm`, split/cat passes); the next sweeps turn these opt-in families on in combination to look for pairs, as proposed. (3) Decode graphs should use a real KV cache. (4) If GPU access appears, the same commands run with `-device cuda` after a one-line change in the model builders.

References

- [1] O. Zinenko et al. The MLIR Transform Dialect. arXiv:2409.03864, 2024.
- [2] A. Vohra et al. Mind the Abstraction Gap: Bringing Equality Saturation to Real-World ML Compilers. OOPSLA 2025.
- [3] Z. Jia et al. TASO. SOSP 2019.
- [4] PyTorch. `torch/_inductor/compiler_bisector.py`, 2025.
- [5] A. Zeller and R. Hildebrandt. Simplifying and Isolating Failure-Inducing Input. IEEE TSE, 2002.
- [6] J. Ansel et al. PyTorch 2. ASPLOS 2024.